Group 2 Activity Packet — Steganography

"The Leaked Memo" — self-run, Judge-led, no teacher intervention needed

How This Document Works

This packet is designed so your group can run the entire activity yourselves, without waiting for the teacher at any point. Follow the stages below IN ORDER — each stage tells you exactly what you receive, what to do with it, and who to hand your result to next.

- The teacher will not check your work step-by-step during the activity. You are expected to self-check as you go (see the yellow boxes) and cross-check with your teammate(s) (see the blue boxes).
- Where you see a blank line, that is YOUR job to work out — the answer is deliberately not printed anywhere in this document. If you get stuck, re-read the relevant section of your Study Material.
- The Judge / Timekeeper (the last stage) is in charge of running your group's clock, confirming the final results match, scoring part of the rubric themselves using the guidance printed in their own stage, and filling in the Result Card at the end — that Result Card is what gets submitted for grading, and the teacher's script totals it automatically.

Do not skip ahead: Only read the stage that applies to your own role. Reading a later stage's instructions before it's your turn defeats the purpose of the activity — you'll be found out anyway, because the numbers won't make sense to you without the steps in between.

The Scenario

A whistle-blower team needs to get a single confirmation letter to an editor team through a monitored file-sharing channel, without the monitoring team realising a message is even being sent. The letter is hidden inside the least significant bits of an 8-pixel thumbnail image, scattered by a secret key. The security monitoring team inspects the image and, suspecting something, must brute-force the small set of possible keys to recover the hidden letter.

Roles in Your Group (10 students)

- Student 1 — Key Generator → Jayanth
- Students 2, 3 — Sender A & B → Nanda, Sebin
- Student 4 — Uploader / Channel → Shreyas
- Students 5, 6 — Receiver A & B → Jibin, Jyothis
- Students 7, 8, 9 — Attacker A, B, C → Hari, Anu, KARTHIK
- Student 10 — Judge / Timekeeper → Shreya

Given Data (the only numbers you start with)

```
Cover pixels: [110, 113, 120, 118, 121, 117, 119, 116]
Message to hide: the single letter 'B'
```

The key, the position order, and the final stego array must all be worked out by the roles below — none of it is given upfront.

Stage-by-Stage Instructions

Stage 1 — Key Generator (Student 1)

You start with: Nothing except this instruction sheet — you start the activity.

What to do:

1. Choose a key that is coprime with 8 (If unsure, use key = 5) — this guarantees every one of the 8 pixels gets used exactly once.

2. Decide the embedding-position formula: position(i) = (key × i + 3) mod 8, for i = 0..7.

Self-check before you hand off: Compute position(i) for all 8 values of i (0 through 7) and list them out. Check that all 8 results are different numbers, covering 0 through 7 exactly once with no repeats. If any number repeats, your key is not valid — pick a different one and redo this check.

Your chosen key and the 8 positions [i=0..7]: 3

Hand off: Share the key and the formula privately — never let the Attacker team see or hear this to the Sender team and Receiver team only. Do not show them this stage's instructions — only give them the value(s) named above.

Stage 2 — Sender (Students 2 & 3)

You receive: The secret key and position formula (privately). You already know the message 'B' and the cover pixels from the Given Data. — handed to you by Key Generator.

What to do:

3. Convert the letter 'B' into 8-bit binary using the ASCII method from your Study Material.
4. Using the position formula, compute which pixel each of the 8 bits should go into.
5. For each bit, overwrite ONLY the least-significant bit (LSB) of that pixel's value with the message bit, keeping every other bit of the pixel unchanged. Build the final 8-pixel stego array this way.

Cross-check with your partner(s): Both of you compute the position order and the final stego array independently, then compare all 8 values before handing it over.

Your final stego pixel array (all 8 values): [110, 112, 120, 118, 120, 117, 119, 116]

Hand off: Hand over the finished stego array only — not your working, not the key to the Uploader. Do not show them this stage's instructions — only give them the value(s) named above.

Stage 3 — Uploader / Channel (Student 4)

You receive: The finished stego pixel array. — handed to you by Sender.

What to do:

6. Pass the stego array to the Receiver team.
7. Do this visibly enough that the Attacker/Monitoring team can copy the array down — this is intentional, it represents a monitored public channel.

Hand off: Deliver the array to the Receiver team (and Attacker team copies it as you pass). Do not show them this stage's instructions — only give them the value(s) named above.

Stage 4 — Receiver — legitimate path (Students 5 & 6)

You receive: The stego array, and the key + formula shared privately earlier by the Key Generator. — handed to you by Uploader.

What to do:

8. Regenerate the same 8 positions using the key and formula.
9. Read the LSB of each pixel in that position order to reconstruct the 8-bit binary string.
10. Convert the 8 bits back into a character.
11. Record how long this step took you, in minutes.

Cross-check with your partner(s): Both of you regenerate the position order and read the bits independently, then compare your recovered character before reporting.

Your recovered letter and time taken (min): 'B' and 4 mins

Hand off: Report your recovered letter and your time to the Judge. Do not show them this stage's instructions — only give them the value(s) named above.

Stage 5 — Attacker — attack path (Students 7, 8 & 9)

You receive: The stego array only. You do NOT have the key — you must find it yourself. — handed to you by what you copied off the Uploader's channel.

What to do:

12. Student 7: list every key from 1 to 7 that is coprime with 8 (produces a valid, non-repeating position order when you check it the same way the Key Generator did).
13. Student 8: for each valid key found, compute its position order and decode the resulting 8 bits into a character.
14. Student 9: look at all the decoded results — identify which ONE result is a plausible, printable, meaningful character rather than noise. Confirm with Students 7 & 8 before reporting.
15. Record your total time and how many candidate keys you had to try before finding the right one.

Cross-check with your partner(s): Student 8 should double check Student 7's list of valid keys before decoding each one; all three of you should agree on which decoded result is the real message before reporting it.

Your recovered letter, total time (min), and number of keys tried:

20 mins and 3 key tried

Hand off: Report your recovered letter, your total time, and how many keys you tried to the Judge. Do not show them this stage's instructions — only give them the value(s) named above.

Stage 6 — Judge / Timekeeper — you lead this group (Student 10)

You receive: Both teams' recovered letters, times, and (from the Attacker) how many keys they tried. — handed to you by Receiver team and Attacker team.

What to do:

16. You are in charge of running this activity for your group — keep time for each stage above and keep everyone moving, without waiting for the teacher.
17. Confirm the Receiver's letter and the Attacker's letter are the SAME. If they differ, send both teams back to recheck their working.
18. Note how many candidate keys the Attacker needed to try.

Score before you fill in the Result Card: As Judge, you also score three rows of the rubric yourself, right now, using the guidance below — there is no separate teacher check for these.

19. **Attacker/Claimant correctness (0-5):** full 5 if they independently reached the same correct answer as the legitimate side; partial 2-3 if their method was right but they ran out of time or made an arithmetic slip; 0 if the method itself was wrong.
20. **Attacker/Claimant efficiency (0-3):** reward a systematic method over guessing — e.g. Group 2 testing all valid keys methodically rather than randomly.
21. **Explanation quality (0-3):** before scoring this, pick ONE member at random from each sub-team (not the strongest student) and ask them to explain one step out loud. Full marks only if they can actually explain it, not just read out a number.

Once scored, fill in the Result Card on the next page completely and legibly.

Result Card — fill this in and submit

The Judge / Timekeeper fills in this card once both the legitimate path and the attack/claim path are complete and confirmed to match. This card is what your teacher will use to grade your group automatically — fill in every field exactly as instructed, using digits/letters only where shown.

GROUP: 2

STUDENT NAMES / ROLL NUMBERS (all 10, by role):

LEGIT_RESULT (Receiver's recovered letter): 'B'

LEGIT_TIME_MIN (Receiver's time, in minutes): 4 mins

ATTACK_RESULT (Attacker's recovered letter): B

ATTACK_TIME_MIN (Attacker's total time, in minutes): 35:25

ATTACK_ATTEMPTS (how many candidate keys the Attacker tried, 1-4):
4

ATTACK_METHOD_NOTE (1-2 lines: how did the Attacker narrow down the right key?):
Tried the offset value 3 with all possible key = {1,3,5,7}, then rearranged the LSB to find the message

ATTACK_RESULT_SCORE (0-5, using the correctness guidance above):
3

ATTACK_EFFICIENCY_SCORE (0-3, using the efficiency guidance above):
4

EXPLANATION_SCORE (0-3, from the random member you asked to explain a step):
3

JUDGE_SUMMARY (1 line: did both match? how many keys did Attacker try?):
The Attackers tried 4 keys and both matched.

How to submit: Hand this page (or a clear photo of it) to your teacher. If asked to submit digitally, type these exact values into a plain text file named Group2_Result.txt, one "FIELD: value" line per field exactly as labelled above, and send it as your teacher instructs.

	Student Name	Roll no:	Role
1)	Jayanth Mathew	14	Key Generator
2)	Nandakishore	24	Sender A
3)	Deon Sebastin	9	Sender B
4)	Shazin Ahmed	28	Uploader
5)	Jibin	15	Reciever A
6)	Jyothis	19	Reciever B
7)	Harikrishnan	13	Attacker C
8)	Karthik Anil	20	Attacker A
9)	Anusree Babu	7	Attacker B
10)	Shaun Joseph Godly	27	Judge/Timekeeper.

Overall time - 1:11:34 Attacker time: 35:25.

Group 2 Judge / Timekeeper — Result Card Guide

"The Leaked Memo" — for the Judge/Timekeeper (Student 10) only

Your Role Recap

- Keep time for every stage and keep the group moving — you run this activity, not the teacher.
- Confirm the Receiver's recovered letter and the Attacker's recovered letter are the SAME before scoring anything. If they differ, send both teams back to recheck their working.
- Score the 3 judgement rows below, then fill in every Result Card field.

Field-by-Field: What To Write On The Result Card

GROUP

Your group number.

```
Example only — NOT your group's real answer:
GROUP: 2
```

STUDENT NAMES / ROLL NUMBERS

List all 10 students by role.

```
Example only — NOT your group's real answer:
STUDENT NAMES / ROLL NUMBERS: Key Gen-Roll 6; Sender A/B-Roll 12,18; Uploader-Roll 1;
Receiver A/B-Roll 8,20; Attacker A/B/C-Roll 5,10,23; Judge-Roll 16
```

LEGIT_RESULT

The single letter the Receiver team reported to you.

```
Example only — NOT your group's real answer:
LEGIT_RESULT: K
```

LEGIT_TIME_MIN

How many minutes it took the Receiver team from getting the stego array to reporting their letter.

```
Example only — NOT your group's real answer:
LEGIT_TIME_MIN: 2
```

ATTACK_RESULT

The letter the Attacker team reported — should match LEGIT_RESULT if they cracked it correctly.

```
Example only — NOT your group's real answer:
ATTACK_RESULT: K
```

ATTACK_TIME_MIN

The Attacker team's TOTAL time, from first seeing the stego array to reporting their letter.

```
Example only — NOT your group's real answer:
ATTACK_TIME_MIN: 6
```

ATTACK_ATTEMPTS

How many of the 4 valid candidate keys they had to try before finding the right one (1-4).

```
Example only — NOT your group's real answer:
ATTACK_ATTEMPTS: 3
```

ATTACK_METHOD_NOTE

1-2 lines on HOW the Attacker team narrowed down the right key — your evidence for the score fields below.

```
Example only — NOT your group's real answer:
ATTACK_METHOD_NOTE: Tried all 4 valid keys in order; two gave unprintable noise, one gave a
stray symbol, one gave a clean printable letter.
```

How To Score The 3 Judgement Rows

These three fields are the only ones where YOU decide the number, using the guidance below — the same guidance a teacher would otherwise apply. Fill these in before you touch the Result Card.

ATTACK_RESULT_SCORE (0-5)

Full 5 if the Attacker independently reached the SAME correct letter as the Receiver, without ever being told the key. Partial 2-3 if their method was right but they ran out of time or made an error. 0 if the method itself was wrong.

ATTACK_EFFICIENCY_SCORE (0-3)

Reward a systematic method over guessing — e.g. testing all 4 valid keys methodically rather than randomly. Give 3 for a clearly systematic approach, 1-2 if they got there haphazardly or skipped a candidate they should have checked, 0 for no real method.

EXPLANATION_SCORE (0-3)

Before writing this score, pick ONE member at random from each sub-team — not the strongest student — and ask them to explain one step out loud. Score 3 only if they can actually explain it (not just read out a number), 1-2 for a partial explanation, 0 if they can't explain it at all.

Do this before you write the number: The explanation check has to happen live, right now, with a student you pick — not the team's own choice of who answers.

Submitting

How to submit: Hand the completed Result Card page (or a clear photo of it) to your teacher. If asked to submit digitally, type the exact values into a plain text file named Group2_Result.txt, one "FIELD: value" line per field, exactly as labelled above.

Jayanth Mathew
14/55

Group-2 Key Generation

$\phi = 8$

(1, 3, 5, 7) → coprime

⇒ let's start with (1, 8)

then we choose the gcd (8)

since gcd(1,8) ⇒ 1 × 8 + [0] ✓

next take (3,8) ⇒ 3 × 2 + [2]

6

gcd 2 ⇒ 1 × 1 + [1]

1

gcd 1 ⇒ 1 × 1 + [0]

gcd(5,8) = 5 × 1 + [3]

gcd 3 = 3 × 2 + [2]

gcd 2 = 1 × 1 + [1]

gcd 1 2 1 × 1 + 0

So hence we can conclude the key as 3

since 5 is also an coprime to 8 but after 3, so we take 3 as key

The key has been generated and has been sented -

Key = 3 Group-2 Sender A

n = 8 Mandakghore V.

offset = 3 CY

S5 / 24

Cover pixel

[110, 113, 120, 118, 121, 117, 119, 116]

Letter to Hide : B

ASCII Value = 66

128	64	32	16	8	4	2	1
0	1	0	0	0	0	1	0

0 1 00 00 1 0

Position (i) = (key × i + offset) mod N.

position (0) = (3 × 0 + 3) mod 8

= (0 + 3) mod 8

= 3 mod 8 = 3 //

position (1) = (3 × 1 + 3) mod 8

= (6) mod 8 = 6 //

position (2) = (3 × 2 + 3) mod 8

= 9 mod 8 = 1 //

position (3) = (3 × 3 + 3) mod 8

= 12 mod 8 = 0 //

position (4) = (3 × 4 + 3) mod 8

= 15 mod 8 = 7 //

position (5) = (3 × 5 + 3) mod 8 = 18 mod 8 = 2 //

position (6) = (3 × 6 + 3) mod 8

= (18 + 3) mod 8

= 21 mod 8 = 5

= 5,

position (7) = (3 × 7 + 3) mod 8

= 24 mod 8 = 0 //

0	1	0	0	0	0	1	0

Pixel Positions [3, 6, 1, 4, 7, 2, 5, 0]

[110, 113, 120, 118, 121, 117, 119, 116]

0 1 2 3 4 5 6 7

118 = 01110110 = 01110110

119 = 01110111 = 01110111

113 = 01110001 = 01110000 = 112 //

121 = 01111001 = 01111000 = 120 //

116 = 01110100 = 01110100

120 = 01111000 = 01111000

117 = 01110101 = 01110101

110 = 01101110 = 01101110

New Pixel =

[110, 112, 120, 118, 120, 117, 119, 116]

Group-2 Sender B Deon Sebastian S5 CY

A. a — 67 · Key = 3. Hide = B. Rollno: 9

B — 66 · Offset = 3. k.

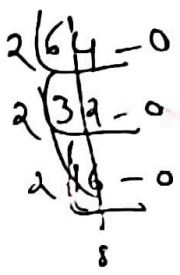
Ascii value = 66.

64 + 2.

0 1 0 0 0 0 1 0

· n = 8

Offset = 1.



[110, 113, 120, 118, 121, 111, 119, 116]

0 1 2 3 4 5 6 7

pixels.

position (i) = (key (i) + offset) Mod N. 128 64 32 16 8 4 2 1.

0 1 0 0 0 0 1 0.

= (0) = (3 × 0) Mod 8

= 2 Mod 8 = 3

= (1) = (3 × 2) Mod 8

= 8 9 mod 8 = 1

Position (0) = (3 × 0 + 3) Mod 8

= 8 + 3 = 3 Mod 8 = 3.

position (1) = (3 × 1 + 3) Mod 8.

= 3 + 3 = 6 Mod 8 = 6

position (2) = (3 × 2 + 3) Mod 8

= 6 + 3 = 9 Mod 8 = 1

position (3) = (3 × 3 + 3) Mod 8.

= 9 + 3 = 12 Mod 8 = 4

position (4) = (3 × 4 + 3) Mod 8.

= 12 + 3 = 15 mod 8 = 7.

position (5) = (3 × 5 + 3) Mod 8.

= 15 + 3 = 18 Mod 8 = 2.

Position(6) = (3×6+3) Mod 8

= 18+3 = 21 Mod 8. = 5.

Position (7) = (3×7+3) Mod 8.

= 21+3 = 24 mod 8 = 0

0	1	0	0	0	0	1	0
3	6	1	4	7	2	5	0
0	1	2	3	4	5	6	7

Index

[110 113 120 118 121 117 119 116]

118 = 01110110 = 01110110

119 = 01110111 = 01110111

113 = 01110001 = 01110000 = 112.

121 = 01111001 = 01111000 = 120

116 = 01110100 = 01110100

120 = 01111000 = 01111000

117 = 01110101 = 01110101

110 = 01101110 = 01101110

(110, 112, 120, 118, 120, 117, 119, 116)

The pixels has been encrypted and given to the uploader.

loaded Group-2 Shazin Ahmmed
95CY
28

[3, 6, 1, 4, 7, 2, 5, 0]

New pixel

[110, 112, 120, 118, 120, 117, 119, 116]

The encrypted key has been given by the sender and is gonna be given to the receiver

Gr

Receiver -A

Name: Jibin V Johnson

Roll No: 15 - ~~Rollno~~

Group II

pixel position

[3, 6, 1, 4, 7, 2, 5, 0]

[110, 112, 120, 118, 120, 117, 119, 116]

0 0 0 0 0 1 1 0

0 1 2 3 4 5 6 7

LSB Even - 0

Odd - 1

(0.1, 0 0 0 0 1 0) = B. (66)

Group-2

Receiver-B

Jyothis Ramod
Cybersecurity
Roll no: 19



[3, 6, 1, 4, 7, 2, 5, 0]

{110, 112, 120, 118, 120, 117, 119, 116}

0	0	0	0	0	1	1	0
0	1	2	3	4	5	6	7

[0 1 0 0 0 0 1 0] = BG#,(B)

it has gotten

- The pixel has been received and decrypted.

Group 2. Student-7

Karthik Giri
Roll no: 20

Attacker A

[110 112 120 118 120 117 119 116]

Numbers from 1 to 7 is considered as the rest are greater than 8

- gcd(x,8) = 1.
- gcd(1,8) = 1
- gcd(2,8) = 2
- gcd(3,8) = 1
- gcd(4,8) = 4
- gcd(5,8) = 1
- gcd(6,8) = 2
- gcd(7,8) = 1
- gcd(8,8) = 8

Possible key values: 1, 3, 5, 7

- Key will be always co-prime of number of pixels

The pixels has been intercepted and has decrypted the message after 3 ~~four~~ attempts.

10 112 120 118 120 117 119 gcd 116] k, 55) CY = # 07

Attacker B

$pos(i) = (i \times k + offset) \bmod N$

gcd possible key values = {1, 3, 5, 7}

Take offeset as 1, and k = 1

i = 0

$pos(0) = (0 \times 1 + 1) \bmod 8$

$= 1 \bmod 8$

$= 1$

$pos(1) = (1 \times 1 + 1) \bmod 8$

$= (1+1) \bmod 8 = 2 \bmod 8$

$= 2$

$pos(2) = (2 \times 1 + 1) \bmod 8$

$= 3 \bmod 8 = 3$

$pos(3) = (3 \times 1 + 1) \bmod 8$

$= 4 \bmod 8 = 4$

$pos(4) = (4 \times 1 + 1) \bmod 8$

$= 5$

$pos(5) = (5 \times 1 + 1) \bmod 8$

$= 6$

$pos(6) = (6 \times 1 + 1) \bmod 8$

$= 7$

$pos(7) = (7 \times 1 + 1) \bmod 8$

$= 0$

k = 7

ie offset = 2 $n > k$

$pos(0) = (0 \times 1 + 2) \bmod 8$

$= 2$

$pos(1) = (1 \times 1 + 2) \bmod 8$

$= 3$

$pos(2) = (2 \times 1 + 2) \bmod 8$

$= 4$

$pos(3) = (3 \times 1 + 2) \bmod 8$

$= 5$

$pos(4) = (4 \times 1 + 2) \bmod 8$

$= 6$

$pos(5) = (5 \times 1 + 2) \bmod 8$

$= 7$

$pos(6) = (6 \times 1 + 2) \bmod 8$

$= 0$

$pos(7) = (7 \times 1 + 2) \bmod 8$

$= 1$

offset = 3

$pos(0) = (0 \times 1 + 3) \bmod 8$

$= 3$

$pos(1) = (1 \times 1 + 3) \bmod 8$

$= 4$

$pos(2) = (2 \times 1 + 3) \bmod 8$

$= 5$

$pos(3) = (3 \times 1 + 3) \bmod 8 = 6$

$pos(4) = (4 \times 1 + 3) \bmod 8 = 7$

$pos(5) = (5 \times 1 + 3) \bmod 8$
$= 0$
$pos(6) = (6 \times 1 + 3) \bmod 8$
$= 1$
$pos(7) = (7 \times 1 + 3) \bmod 8$
$= 2$

$pos(5) = (5 \times 3 + 3) \bmod 8$
$= 2$
$pos(6) = (6 \times 3 + 3) \bmod 8$
$= 5$
$pos(7) = (7 \times 3 + 3) \bmod 8$
$= 0$

key = 3 offset = 1
$pos(0) = (0 \times 3 + 1) \bmod 8$
$= 1$
$pos(1) = (1 \times 3 + 1) \bmod 8$
$= 4$
$pos(2) = (2 \times 3 + 1) \bmod 8$
$= 7$
$pos(3) = (3 \times 3 + 1) \bmod 8$
$= 2$
$pos(4) = (4 \times 3 + 1) \bmod 8$
$= 5$
$pos(5) = (5 \times 3 + 1) \bmod 8$
$= 0$
$pos(6) = (6 \times 3 + 1) \bmod 8$
$= 3$
$pos(7) = (7 \times 3 + 1) \bmod 8$
$= 6$

key = 5 offset = 3
$pos(0) = (0 \times 5 + 3) \bmod 8 = 5$
$pos(1) = (1 \times 5 + 3) \bmod 8 = 0$
$pos(2) = (2 \times 5 + 3) \bmod 8 = 5$
$pos(3) = (3 \times 5 + 3) \bmod 8 = 2$
$pos(4) = (4 \times 5 + 3) \bmod 8 = 7$
$pos(5) = (5 \times 5 + 3) \bmod 8 = 4$
$pos(6) = (6 \times 5 + 3) \bmod 8 = 1$
$pos(7) = (7 \times 5 + 3) \bmod 8 = 6$

key = 3 offset = 3
$pos(0) = (0 \times 3 + 3) \bmod 8$
$= 3$
$pos(1) = (1 \times 3 + 3) \bmod 8$
$= 6$
$pos(2) = (2 \times 3 + 3) \bmod 8$
$= 1$
$pos(3) = (3 \times 3 + 3) \bmod 8$
$= 4$
$pos(4) = (4 \times 3 + 3) \bmod 8 = 7$

key = 7 offset = 3
$pos(0) = (0 \times 7 + 3) \bmod 8 = 3$
$pos(1) = (1 \times 7 + 3) \bmod 8 = 2$
$pos(2) = (2 \times 7 + 3) \bmod 8 = 1$
$pos(3) = (3 \times 7 + 3) \bmod 8 = 0$
$pos(4) = (4 \times 7 + 3) \bmod 8 = 7$
$pos(5) = (5 \times 7 + 3) \bmod 8 = 6$
$pos(6) = (6 \times 7 + 3) \bmod 8 = 5$
$pos(7) = (7 \times 7 + 3) \bmod 8 = 4$

0 1 2 3 4 5 6 7

0 112 120 118 120 117 119 116

01110 01111000 01111000 01110101 01110111 01110100

01110000 01110110

Take of = 1 k=1 ⇒ pos 1, 2, 3, 4, 5, 6, 7, 8 ⇒ 00000100 ⇒ 6

6 is not readable in ASCII ∴ key 1 is not key

(9:36)

OF=1 k=1 ⇒ pos 2, 3, 4, 5, 6, 7, 8, 1 ⇒ 00111000 ⇒ 56

(16) ⇒ 56 is 9 in ASCII

OF=3 k=1 ⇒ pos 3, 4, 5, 6, 7, 8, 1, 2 ⇒ 00110000 ⇒ 48

(18:10) ⇒ 48 is 1 in ASCII

OF=1 k=3 ⇒ pos 1, 4, 7, 2, 5, 8, 3, 6 ⇒ 00001001 ⇒ 9

⇒ 9 is not readable in ASCII

OF=3 K=3 ⇒ pos 3, 6, 1, 4, 7, 2, 5, 8 ⇒ 01000010 ⇒ 66

⇒ 66 is B in ASCII

OF=3 k=5 ⇒ pos 3, 8, 5, 2, 7, 4, 1, 6 ⇒ 00100001 ⇒ 17

⇒ 17 not readable in ASCII

OF=3 k=7 ⇒ pos 3, 2, 1, 8, 7, 6, 5, 4 ⇒ 00000110 ⇒ 6

⇒ 6 not readable in ASCII

When offset is 3, we get 2^2 possible outcomes, ⇒ for keys 1 and 3

1 ⇒ 1 in ASCII

3 ⇒ B in ASCII